# Setup & Prerequisites

This vignette covers everything you need **before** using `snowflakeR` – system requirements, Python setup, authentication, and environment-specific configuration. For usage, see `vignette("getting-started")`.

## Quick reference

| Requirement | Version |
|---|---|
| R | >= 4.1.0 |
| Python | >= 3.9 (3.11 recommended) |
| `snowflake-ml-python` | >= 1.5.0 |
| `snowflake-snowpark-python` | (latest) |
| `reticulate` | >= 1.34 |

## 1. System requirements

### R

Install R >= 4.1.0 from CRAN or via your system package manager. `snowflakeR` uses the base pipe (`|>`) and other features introduced in R 4.1.

### Python

`snowflakeR` uses `reticulate` to call the Snowflake ML Python SDK under the hood. You need Python >= 3.9 available on your system. We recommend Python 3.11 for best compatibility with `snowflake-ml-python`.

### Conda (recommended) or pip

We strongly recommend using **conda** (or `micromamba`) to manage the Python environment. This avoids version conflicts between `snowflake-ml-python`, `pandas`, `pyarrow`, and other dependencies. `pip` works fine too if you prefer virtualenvs.

## 2. Installing snowflakeR

```r
# Install from GitHub
# install.packages("pak")
pak::pak("Snowflake-Labs/snowflakeR")
```

## 3. Setting up the Python environment

Choose **one** of the following approaches.

#### Option A: Automatic (recommended for most users)

```r
library(snowflakeR)
sfr_install_python_deps()
```

This creates a conda environment called `r-snowflakeR` with Python 3.11 and all required packages. After running this once, `reticulate` will automatically find and use the environment.

#### Option B: Conda (manual)

```bash
conda create -n r-snowflakeR python=3.11 -y
conda activate r-snowflakeR
pip install snowflake-ml-python snowflake-snowpark-python
```

Then tell R to use it:

```r
# In your .Renviron or at the top of your script
Sys.setenv(RETICULATE_PYTHON = "~/.conda/envs/r-snowflakeR/bin/python")

# Or in R:
reticulate::use_condaenv("r-snowflakeR")
```

#### Option C: pip / virtualenv

```bash
python3.11 -m venv ~/.virtualenvs/r-snowflakeR
source ~/.virtualenvs/r-snowflakeR/bin/activate
pip install snowflake-ml-python snowflake-snowpark-python
```

```r
reticulate::use_virtualenv("~/.virtualenvs/r-snowflakeR")
```

#### Option D: System Python

If you install the Snowflake packages into your system Python:

```bash
pip install snowflake-ml-python snowflake-snowpark-python
```

No extra `reticulate` configuration needed – it will find your system Python automatically.

## 4. Verifying the setup

After installing `snowflakeR` and setting up Python, verify everything works:

```r
library(snowflakeR)
sfr_check_environment()
#> R version: "4.3.1"
#> v reticulate available
#> Python: "/path/to/python"
#> v snowflake-ml-python available
```

This checks:

- R version
- `reticulate` availability
- Python discovery
- `snowflake-ml-python` import
- `snowflakeauth` availability (optional)

## 5. Authentication

`snowflakeR` supports multiple authentication methods. Choose the one that fits your environment.

### connections.toml (recommended for local use)

Create `~/.snowflake/connections.toml`:

```
[default]
account = "xy12345.us-east-1"
user = "MYUSER"
authenticator = "externalbrowser"
warehouse = "COMPUTE_WH"
database = "MY_DB"
schema = "PUBLIC"

[production]
account = "xy12345.us-east-1"
user = "SVC_USER"
private_key_path = "~/.snowflake/rsa_key.p8"
warehouse = "PROD_WH"
database = "PROD_DB"
schema = "ML"
```

Then connect with:

```
conn <- sfr_connect()                      # uses [default]
conn <- sfr_connect(name = "production") # uses [production]
```

If `snowflakeauth` is installed, `snowflakeR` uses it for credential resolution. If not, `snowflakeR` reads the TOML file directly.

### Key-pair authentication

Generate a key pair:

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```

Register the public key with your Snowflake user, then:

```r
conn <- sfr_connect(
  account          = "xy12345.us-east-1",
  user             = "MYUSER",
  private_key_file = "~/.snowflake/rsa_key.p8"
)
```

This is the recommended method for CI/CD and automated workflows.

#### External browser (SSO)

```r
conn <- sfr_connect(
  account       = "xy12345.us-east-1",
  user          = "MYUSER",
  authenticator = "externalbrowser"
)
```

Opens your browser for SSO. Good for interactive local sessions.

#### Workspace Notebooks (auto-detect)

No configuration needed. See the Workspace Notebooks section below.

## 6. Environment-specific setup

#### RStudio / Posit Workbench

This is the most common setup. After steps 2-3 above, configure `reticulate` to find your Python:

**Option 1: `.Renviron` file** (recommended – persists across sessions)

Create or edit `~/.Renviron`:

```
RETICULATE_PYTHON=~/.conda/envs/r-snowflakeR/bin/python
```

Restart R after editing.

**Option 2: RStudio Python settings**

Go to **Tools > Global Options > Python** and select the `r-snowflakeR` interpreter.

**Option 3: Per-script**

```r
reticulate::use_condaenv("r-snowflakeR")
library(snowflakeR)
```

> **Note**: `reticulate` locks the Python version on first use in an R session. Make sure to configure Python *before* calling any `snowflakeR` functions.

### VS Code

1. Install the R Extension for VS Code.
2. Set the Python path in your workspace `.Renviron` or in VS Code settings:

```
{
  "r.rterm.option": ["--no-save", "--no-restore"],
  "python.defaultInterpreterPath": "~/.conda/envs/r-snowflakeR/bin/python"
}
```

3. For Jupyter notebooks in VS Code with an R kernel, install `IRkernel`:

```
IRkernel::installspec()
```

The `.Renviron` approach works the same as in RStudio.

### Jupyter Notebooks (local)

There are two ways to use `snowflakeR` in local Jupyter notebooks:

#### Option A: R kernel (native R)

Install `IRkernel` in your R environment:

```
install.packages("IRkernel")
IRkernel::installspec(user = TRUE)
```

Then create a notebook with the R kernel. `snowflakeR` code runs natively:

```
library(snowflakeR)
conn <- sfr_connect()
```

Make sure the R kernel can find Python by setting `RETICULATE_PYTHON` in `~/.Renviron` before launching Jupyter.

#### Option B: Python kernel + rpy2

This mirrors the Workspace Notebook pattern. Install `rpy2` in your Python environment:

```
pip install rpy2
```

In a Python notebook:

```
%load_ext rpy2.ipython
```

```
%%R
library(snowflakeR)
conn <- sfr_connect()
```

### Snowflake Workspace Notebooks

Workspace Notebooks are managed Jupyter environments inside Snowflake. They run a **Python kernel** – R is available via `rpy2` and `%%R` magic cells.

The R environment is installed by the `sfnb-multilang` toolkit, which handles micromamba, R, rpy2, and package installation. The `snowflakeR` package ships demo notebooks in `inst/notebooks/` – find them with:

```
system.file("notebooks", package = "snowflakeR")
```

Upload the notebook(s) you need to your Workspace and follow the instructions in the first cell. Key notebooks:

| Notebook | Description |
|---|---|
| `workspace_quickstart.ipynb` | Connection, config, queries, ggplot2 |
| `workspace_model_registry.ipynb` | Log, deploy, and serve R models |
| `workspace_feature_store.ipynb` | Feature Store: entities, views, training data |
| `workspace_forecasting_demo.ipynb` | Time series forecasting with forecast + ggplot2 |

**Key differences from local:**

| Aspect | Local | Workspace |
|---|---|---|
| R kernel | Native R | Python + `%%R` |
| Python/R env | You manage | `sfnb_setup.install_r()` |
| Authentication | `connections.toml` / key-pair | Auto-detected (SPCS OAuth) |
| Package install | `install.packages()` | YAML config + `install.packages()` |
| Execution context | Usually set by connection | Must set explicitly (docs) |
| Plotting | Native plot pane | `%%R -w W -h H` + `print(p)` |
| Session lifecycle | Persistent | Re-setup each session (~1-2 min) |

**Setup steps:**

1. **Install R and register `%%R` magic** (run once per session, ~1-2 minutes):

```
# Python cell
from sfnb_setup import install_r
install_r()
```

Each notebook directory includes `sfnb_setup.py`, which auto-detects the environment and installs `sfnb-multilang` from PyPI if needed. For advanced configurations (ADBC, custom packages, Scala/Julia), pass keyword arguments or a YAML config: `install_r(config="configs/snowflaker.yaml")`.

2. **Install and load `snowflakeR`:**

```
%%R
options(repos = c(CRAN = "https://cloud.r-project.org"))

if (nzchar(Sys.getenv("SNOWFLAKER_PATH"))) {
  install.packages(Sys.getenv("SNOWFLAKER_PATH"), repos = NULL, type = "source")
```

```r
} else {
  install.packages("pak", type = "source", quiet = TRUE)
  pak::pak("Snowflake-Labs/snowflakeR", ask = FALSE, upgrade = FALSE)
}

library(snowflakeR)
conn <- sfr_connect()

conn <- sfr_load_notebook_config(conn)
```

All table references should use fully qualified names via `sfr_fqn()`:

```r
sfr_write_table(conn, sfr_fqn(conn, "MY_TABLE"), my_data)
```

See `vignette("workspace-notebooks")` for the full Workspace guide including output helpers, authentication details, and Python-R data exchange.

#### Docker / CI / headless environments

For automated pipelines, containers, or GitHub Actions:

1. Install R and Python in your Dockerfile or CI step.
2. Use **key-pair authentication** (no browser available for SSO).
3. Set environment variables:

```bash
export SNOWFLAKE_ACCOUNT="xy12345.us-east-1"
export SNOWFLAKE_USER="SVC_USER"
export RETICULATE_PYTHON="/usr/bin/python3"
```

4. Or use a `connections.toml` mounted into the container.

For CI that only runs unit tests (no Snowflake connection needed), the Python environment just needs `snowflake-ml-python` installed so that `reticulate` can import the modules. The offline unit tests use mock objects.

See `.github/workflows/R-CMD-check.yml` in the repo for a working GitHub Actions example.

## 7. Optional dependencies

These R packages are optional but enhance `snowflakeR`:

| Package | What it enables |
|---|---|
| `RSnowflake` | DBI-compliant database access (`dbGetQuery`, `dbWriteTable`, `dbplyr`, Connections Pane). See RSnowflake. |
| `snowflakeauth` | `connections.toml` / `config.toml` credential management |
| `httr2` | Pure-R REST inference via `sfr_predict_rest()` |
| `knitr` + `rmarkdown` | Building vignettes from source |

`snowflakeR` provides SQL querying via `sfr_query()` and `sfr_execute()` using the Python/Snowpark bridge. For full DBI compliance, `dbplyr` integration, and the RStudio Connections Pane, install `RSnowflake` and use `sfr_dbi_connection()` to bridge from an `sfr_connection`:

```
dbi_con <- sfr_dbi_connection(conn)
DBI::dbGetQuery(dbi_con, "SELECT 1")

library(dplyr)
tbl(dbi_con, "MY_TABLE") |> filter(score > 90) |> collect()
```

Install all optional dependencies at once:

```
pak::pak("Snowflake-Labs/snowflakeR", dependencies = TRUE)
```

## 8. Troubleshooting

| Issue | Solution |
|---|---|
| `reticulate` can't find Python | Set `RETICULATE_PYTHON` in `.Renviron` or call `reticulate::use_condaenv()` |
| `snowflake.ml` import fails | Run `sfr_install_python_deps()` or `pip install snowflake-ml-python` |
| Python version mismatch | Ensure Python >= 3.9; check with `reticulate::py_config()` |
| `sfr_connect()` hangs | If using `externalbrowser`, check your browser opened a Snowflake tab |
| `sfr_connect()` fails with "account required" | Set up `connections.toml` or pass `account` explicitly |
| `reticulate` uses wrong Python | Call `reticulate::use_condaenv()` *before* loading `snowflakeR` |
| R packages missing in Workspace | Add to YAML config or install interactively via `install.packages()` |
| `%%R` not recognised in Workspace | Run `%load_ext rpy2.ipython` in a Python cell |
| Conda solve takes forever | Use `pip install` inside a conda env (see Option B above) |

## Next steps

- `vignette("getting-started")` – Connecting, queries, DBI/dbplyr
- `vignette("model-registry")` – Logging and deploying R models
- `vignette("feature-store")` – Feature management and training data
- `vignette("workspace-notebooks")` – Full Workspace Notebook guide